



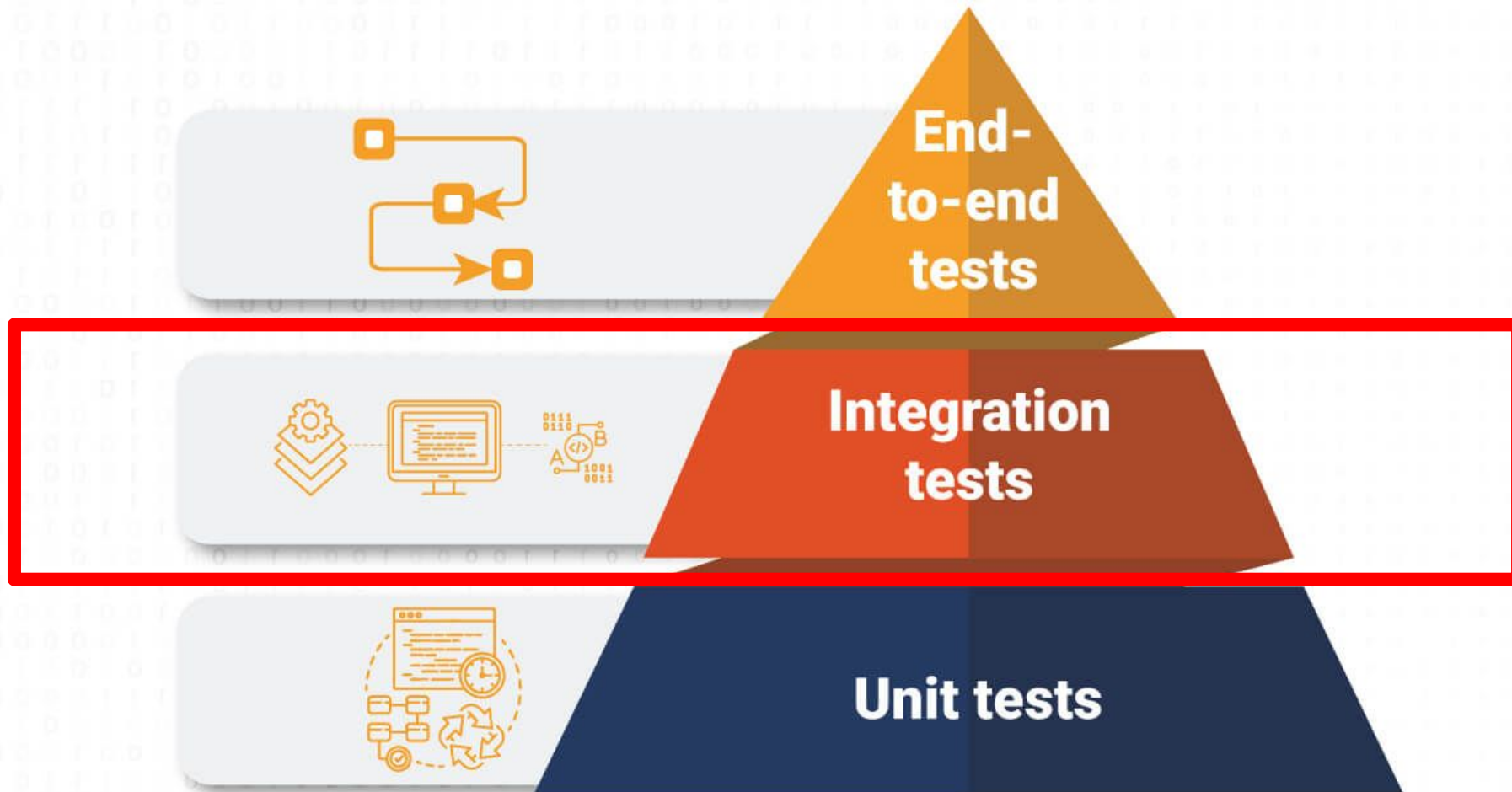
Integration Tests



About me

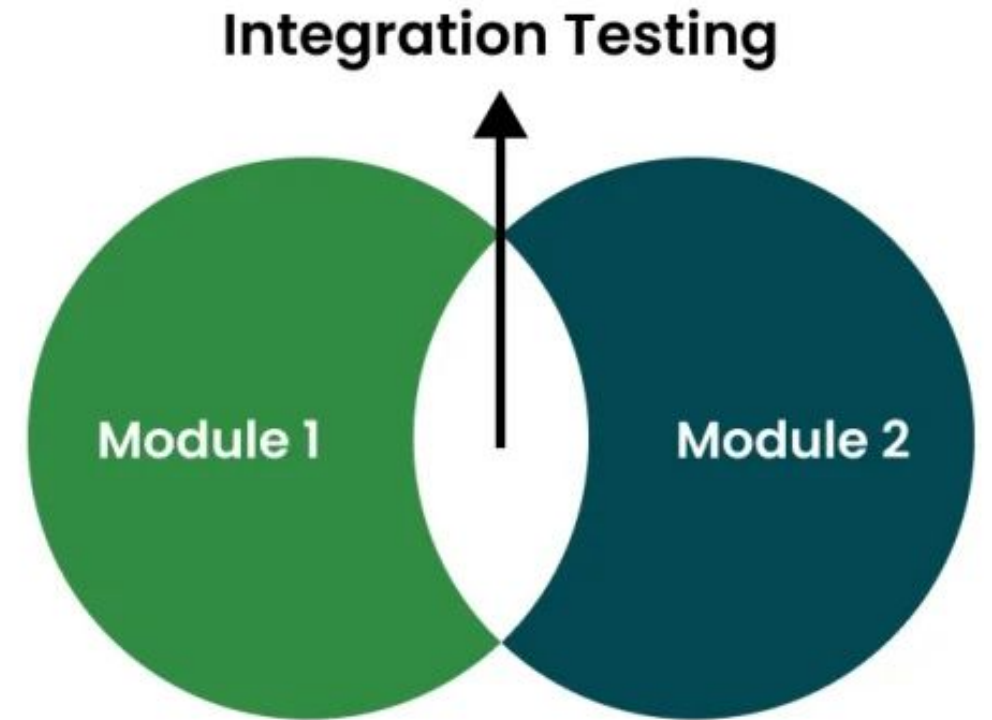
- Name: Vitalii Vasylykha
- Software engineer

What is Integration testing?



What is Integration testing?

Integration testing refers to the process in which the interface of different applications, modules or components of a software application are tested as a combined group



What is Integration testing?

Aspect	Unit Testing	Integration Testing	End-to-End (E2E) Testing
Scope	Individual unit/component (e.g., function, class, module)	Multiple modules interacting	Entire application workflow from start to finish (user perspective)
Purpose	Verify individual components function correctly in isolation	Ensure different modules and interfaces work correctly together	Validate the complete user journey and overall system behavior
Environment	Development environment (often local)	Integration or QA test environment	Staging/Production-like environment
Dependencies	Uses mock data/objects to isolate the unit	Uses real test data; interacts with real, non-mocked systems (e.g., test database)	Uses data that is a copy of real production data; interacts with all real systems and network dependencies
Speed	Instant	Moderate	Long-running
Execution	Automated (typically run on every build)	Automated or manual	Primarily automated to replicate user scenarios
Bugs Caught	Catches bugs early at the root level	Catches interface and data flow issues between components	Catches issues that only appear when the entire system operates together



Where Integration tests add value

Database integration (ORM, transactions)

Database migrations and schema validation

Event-driven systems

Application configuration and dependency injection

HTTP clients and external API

Payment processing flows



Where Integration tests add value

**Database
integration**

Catches: transaction boundary issues, constraint violations, N+1 queries

**Event-driven
systems**

Catches: Message ordering, serialization issues, dead letter queues

**HTTP clients
and external
API**

Catches: Timeout configs, retry logic, error handling



When Integration tests are not the right choice

Pure business logic

Simple input/output validation

UI interaction



Naming convention for Integration test classes

Suffix-based approach

Pattern	Example	Context & Usage
[...]IntegrationTest	UserRepositoryIntegrationTest	-
[...]IT	UserRepositoryIT	Default pattern for the Maven Failsafe Plugin
[...]IntTest	UserRepositoryIntTest	-



Naming convention for Integration test method

Should pattern

Pattern	Example	Context & Usage
should[Expected]When[Condition]	void shouldFailWhenProductIsOutOfStock():	Primarily used for Edge Cases, error scenarios, or specific business rules.
should[Expected]	void shouldCreateOrder():	Typically used for the Happy Path where the context is implied as "standard" or "valid".

Note: Always write the name so that it completes a logical sentence: "The system should [return 404] when [user ID is negative]."



Best Practice 1 - Test Isolation & Independence

Tests should be able to run in any order, or even in parallel, without failing due to shared state.



Best Practice 1 - Test Isolation & Independence. Problem

- Shared static state: Test A sets lastOrderId, Test B expects clean state
- Database records not cleaned: Previous test leaves data, breaks assertions
- Singleton dependencies: Cached instances leak between tests

```
1 @SpringBootTest
2 class OrderServiceIT {
3
4     private static Long lastOrderId; // ❌ Shared state
5
6     @Autowired
7     private OrderRepository orderRepository;
8
9     @Autowired
10    private OrderService orderService;
11
12    @Test
13    void shouldCreateOrder() {
14        Order order = orderService.create("customer@example.com", 100.0);
15        lastOrderId = order.getId(); // ❌ Storing in static field
16
17        assertThat(order.getStatus()).isEqualTo("PENDING"); // ❌ Order remains in DB
18    }
19
20    @Test
21    void shouldCancelOrder() {
22        orderService.cancel(lastOrderId); // ❌ Depends on previous test
23
24        Order order = orderRepository.findById(lastOrderId).orElseThrow();
25        assertThat(order.getStatus()).isEqualTo("CANCELLED");
26    }
27
28    @Test
29    void shouldHaveOnePendingOrder() {
30        long count = orderRepository.countByStatus("PENDING");
31        assertThat(count).isEqualTo(1); // ❌ Depends on test execution order
32    }
33 }
```



Best Practice 1 - Test Isolation & Independence. Fix

- Shared state: Remove static fields, create data per test
- Database pollution: Add `@Transactional` for auto-rollback
- Test interdependence: Each test creates its own data
- Cleanup: Use `@BeforeEach` `deleteAll()` or `@Sql` scripts
- ID assumptions: Don't assert on specific IDs, use `isNotNull()`

```
1 @SpringBootTest
2 @Transactional // ✓ FIX 1: Auto-rollback after each test
3 class OrderServiceIT {
4
5     @Autowired
6     private OrderRepository orderRepository;
7
8     @Autowired
9     private OrderService orderService;
10
11     @Test
12     void shouldCreateOrder() {
13         // ✓ FIX 2: Create data inside test
14         Order order = orderService.create("customer@example.com", 100.0);
15
16         assertThat(order.getStatus()).isEqualTo("PENDING");
17     }
18
19     @Test
20     void shouldCancelOrder() {
21         // ✓ FIX 3: Each test is independent
22         Order order = orderService.create("customer@example.com", 100.0);
23
24         orderService.cancel(order.getId());
25
26         Order cancelled = orderRepository.findById(order.getId()).orElseThrow();
27         assertThat(cancelled.getStatus()).isEqualTo("CANCELLED");
28     }
29
30     @Test
31     void shouldHaveOnePendingOrder() {
32         // ✓ FIX 4: Explicit test data setup
33         orderService.create("customer1@example.com", 100.0);
34         orderService.create("customer2@example.com", 200.0);
35         orderService.create("customer3@example.com", 300.0);
36
37         long count = orderRepository.countByStatus("PENDING");
38         assertThat(count).isEqualTo(3);
39     }
40 }
```


Best Practice 2 - Infrastructure Realism

If you use PostgreSQL in Production, use PostgreSQL in your integration tests.
Don't let an In-Memory DB hide your bugs.



Best Practice 2 - Infrastructure Realism. Problem

- H2 passes but PostgreSQL fails - Different SQL dialect support
- In-memory DB hides constraint issues - UNIQUE constraints behave differently
- Version mismatch - Testing on PostgreSQL 12, production runs 15

```
1 @DataJpaTest
2 @AutoConfigureTestDatabase(replace = Replace.ANY) // ❌ Replaces real DB with H2
3 class ProductRepositoryIT {
4
5     @Autowired
6     private ProductRepository productRepository;
7
8     @Test
9     void shouldFindProductsByCategory() {
10         // ❌ PostgreSQL-specific syntax that works differently in H2
11         Product product = new Product();
12         product.setName("Laptop");
13         product.setPrice(new BigDecimal("999.99"));
14         product.setMetadata("{\"brand\": \"Dell\"}"); // JSONB column in PostgreSQL
15         productRepository.save(product);
16
17         // ❌ JSONB operators work in PostgreSQL but not in H2
18         List<Product> products = productRepository.findByJsonPath("$.brand", "Dell");
19
20         assertThat(products).hasSize(1);
21         // ✅ Passes in H2
22         // ❌ Fails in PostgreSQL - different JSON handling
23     }
24
25     @Test
26     void shouldHandleCaseInsensitiveSearch() {
27         Product product = new Product();
28         product.setName("MacBook Pro");
29         productRepository.save(product);
30
31         // ❌ ILIKE is PostgreSQL-specific, H2 uses different syntax
32         List<Product> products = productRepository
33             .findByNameContainingIgnoreCase("macbook");
34
35         assertThat(products).hasSize(1);
36         // ✅ May pass in H2 with different behavior
37         // ❌ Actual PostgreSQL behavior differs
38     }
39 }
```

Best Practice 2 - Infrastructure Realism. Fix

- H2/In-memory DB → Testcontainers with real PostgreSQL
- Version mismatch → Pin exact production version (postgres:15-alpine)
- SQL dialect differences → Use real database for integration tests
- Missing DB features → Test JSONB, full-text search, constraints on real DB
- Schema differences → Run Flyway/Liquibase migrations in tests

```
1 @SpringBootTest
2 @Testcontainers // Use real PostgreSQL container
3 @Transactional
4 class ProductRepositoryIT {
5
6     @Container
7     static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15-alpine");
8
9     @DynamicPropertySource
10    static void configureProperties(DynamicPropertyRegistry registry) {
11        // Configure Spring to use Testcontainers PostgreSQL
12        .....
13    }
14
15    @Autowired
16    private ProductRepository productRepository;
17
18    @Test
19    void shouldFindProductsByCategory() {
20        // ✓ Test against real PostgreSQL with JSONB support
21        Product product = new Product();
22        product.setName("Laptop");
23        product.setPrice(new BigDecimal("999.99"));
24        product.setMetadata("{\"brand\": \"Dell\"}");
25        productRepository.save(product);
26
27        // ✓ Real PostgreSQL JSONB operators work correctly
28        List<Product> products = productRepository.findByJsonPath("$.brand", "Dell");
29
30        assertThat(products).hasSize(1);
31        assertThat(products.get(0).getName()).isEqualTo("Laptop");
32    }
33
34    @Test
35    void shouldHandleCaseInsensitiveSearch() {
36        Product product = new Product();
37        product.setName("MacBook Pro");
38        productRepository.save(product);
39
40        // ✓ PostgreSQL ILIKE works as expected
41        List<Product> products = productRepository
42            .findByNameContainingIgnoreCase("macbook");
43
44        assertThat(products).hasSize(1);
45        // ✓ Verified against real PostgreSQL behavior
46    }
47 }
```

Best Practice 3 - Stable External Boundaries

Your tests should be able to run offline.
If an external API goes down, your CI/CD pipeline shouldn't.

**Mock Third-Party
APIs**



Best Practice 3 - Stable External Boundaries. Problem

```
1 @SpringBootTest
2 class PaymentGatewayIT {
3
4     @Autowired
5     private PaymentGatewayClient paymentClient;
6
7     @Test
8     void shouldProcessPayment() {
9         PaymentRequest request = new PaymentRequest("USD", 100.00);
10
11         PaymentResponse response = paymentClient.processPayment(request); // ✗ Hits real external API over internet
12
13         assertThat(response.isSuccessful()).isTrue(); // ✗ Fails if 3rd-party API is down or CI lacks internet
14     }
15 }
```



Best Practice 3 - Stable External Boundaries

```
1 @SpringBootTest(properties = "external.payment-api.url=http://localhost:${wiremock.server.port}") // ✅ FIX 1: Route traffic to local mock
2 @AutoConfigureWireMock(port = 0) // ✅ FIX 2: Spin up local WireMock server
3 class PaymentGatewayIT {
4
5     @Autowired
6     private PaymentGatewayClient paymentClient;
7
8     @Test
9     void shouldProcessPayment() {
10         // ✅ FIX 3: Stub external boundary response locally
11         stubFor(post(urlEqualTo("/v1/payments"))
12             .willReturn(aResponse()
13                 .withStatus(200)
14                 .withHeader("Content-Type", "application/json")
15                 .withBody("{\"status\": \"SUCCESS\", \"transactionId\": \"txn_123\" }")));
16
17         PaymentRequest request = new PaymentRequest("USD", 100.00);
18
19         PaymentResponse response = paymentClient.processPayment(request);
20
21         assertThat(response.isSuccessful()).isTrue();
22
23         // ✅ FIX 4: Verify client actually called the mock exactly once
24         verify(1, postRequestedFor(urlEqualTo("/v1/payments")));
25     }
26 }
```



Tools for Mocking Third-Party APIs - Java (WireMock)

```
1 @WireMockTest(httpPort = 8099)
2 class WeatherServiceClientIT {
3
4     @Autowired
5     private WeatherServiceClient weatherClient;
6
7     @Test
8     void shouldFetchWeatherForecastSuccessfully() {
9         // 1. Stub the external boundary
10        stubFor(get(urlPathEqualTo("/api/v1/forecast"))
11                .withQueryParam("city", equalTo("Kyiv"))
12                .willReturn(aResponse()
13                        .withStatus(200)
14                        .withHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
15                        .withBody("""
16                                {
17                                    "temperature": 22.5,
18                                    "condition": "Sunny"
19                                }
20                                """))));
21
22        // 2. Execute the service method
23        WeatherResponse response = weatherClient.getForecast("Kyiv");
24
25        // 3. Assert the mapped response
26        assertThat(response.getTemperature()).isEqualTo(22.5);
27        assertThat(response.getCondition()).isEqualTo("Sunny");
28
29        // 4. Verify the HTTP client made exactly one GET request with the correct parameters
30        verify(1, getRequestedFor(urlPathEqualTo("/api/v1/forecast"))
31                .withQueryParam("city", equalTo("Kyiv")));
32    }
33
34 }
```



Tools for Mocking Third-Party APIs

JavaScript

MSW (Mock Service Worker)

Nock

Mockttp

Python

Responses

respx

httpretty



Best Practice 4 - Execution Speed & Context Optimization

Fast tests are run often. Slow tests are skipped.
Optimize your context startup to keep developers productive.

Maximize
Context
Caching

Lean Test Slices



Best Practice 4 - Execution Speed & Context Optimization. Problem

```
1 @SpringBootTest // ❌ Loads EVERYTHING (Web, Security, Queues) just to test a DB query
2 @DirtiesContext(classMode = ClassMode.AFTER_EACH_TEST_METHOD) // ❌ Destroys Spring context cache
3 class UserRepositoryIT {
4
5     @Autowired
6     private UserRepository userRepository;
7
8     @Test
9     void shouldFindUserByEmail() {
10         userRepository.save(new User("test@example.com"));
11
12         User found = userRepository.findByEmail("test@example.com");
13
14         assertThat(found).isNotNull();
15     }
16 }
```



Best Practice 4 - Execution Speed & Context Optimization. Fix

```
1 @DataJpaTest // ✅ FIX 1: Lean Slice. Loads ONLY database & JPA beans
2 // ✅ FIX 2: No @DirtiesContext. @DataJpaTest automatically rolls back DB transactions after each test, keeping context cached
3 class UserRepositoryIT {
4
5     @Autowired
6     private UserRepository userRepository;
7
8     @Test
9     void shouldFindUserByEmail() {
10         userRepository.save(new User("test@example.com"));
11
12         User found = userRepository.findByEmail("test@example.com");
13
14         assertThat(found).isNotNull();
15     }
16 }
```



Resource

- Грокаємо тестування з Spring
- Integration Testing - Software Engineering
- Best Practices of Integration Testing



Thank you

- Author: Vitalii Vasylykha
- My LinkedIn: [vvasylykha](#)
- Date: June 2026
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

